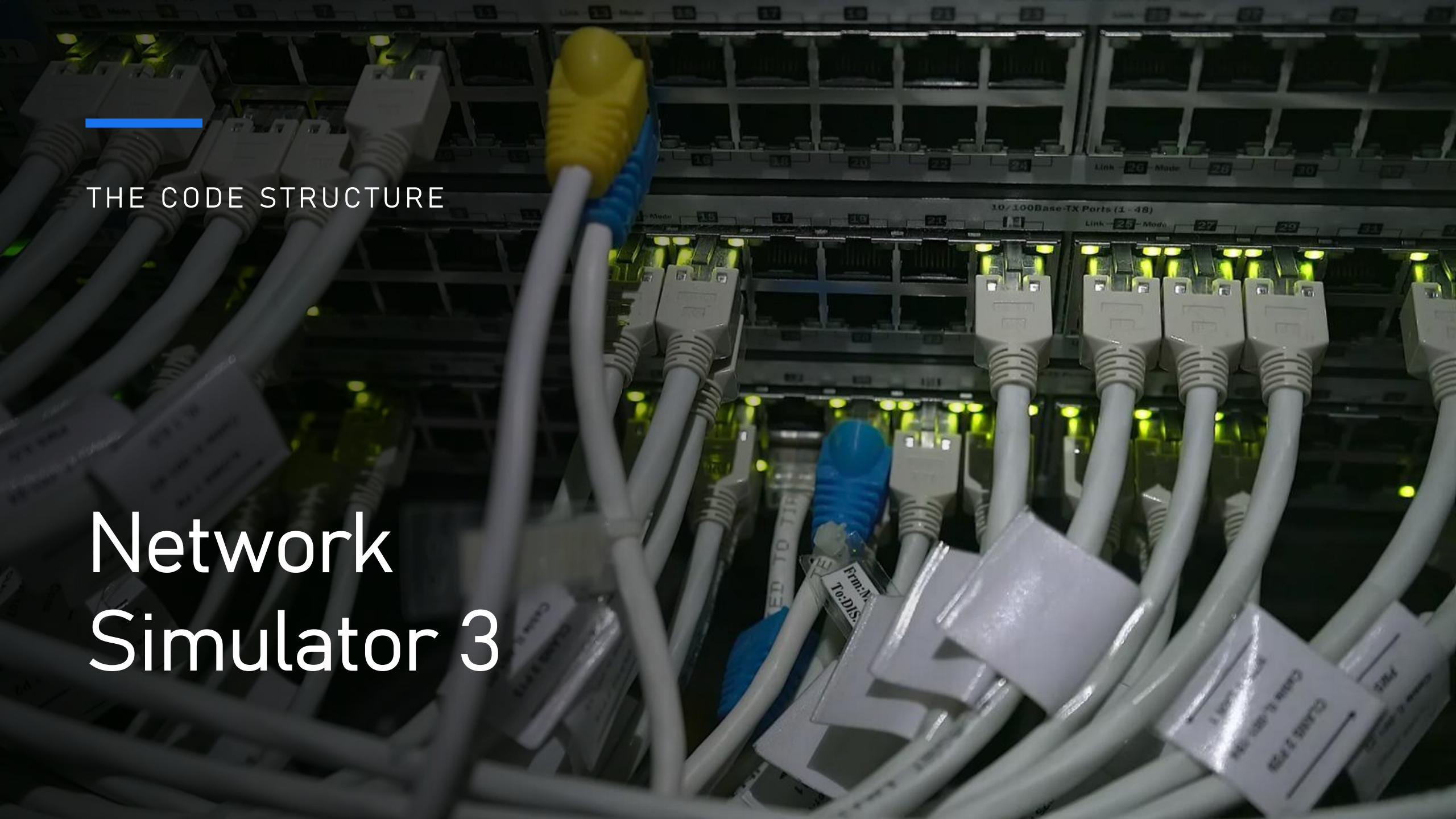




THE CODE STRUCTURE

Network Simulator 3

Understanding the ns-3 Directory Structure



- ns-3.xx/: core source code and modules
- Build/ : Build output directory
- Netanim-3.xxx/ : Network Animator module
- examples/ (inside ns-3.xx/): sample simulation program
- Scratch/: Space for user scripts and simulations
- src/: Drivers, protocols, network models
- doc/ : User and Developer documentation



Program Structure

```
/*  
 * This program is free software; you can redistribute it and/or modify  
 * it under the terms of the GNU General Public License version 2 as  
 * published by the Free Software Foundation;  
 *  
 * This program is distributed in the hope that it will be useful,  
 * but WITHOUT ANY WARRANTY; without even the implied warranty of  
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the  
 * GNU General Public License for more details.  
 *  
 * You should have received a copy of the GNU General Public License  
 * along with this program; if not, write to the Free Software  
 * Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307 USA  
 */
```

The ns-3 simulator is licensed using the GNU General Public License.



-
- The code proper starts with a number of include statements.

```
#include "ns3/core-module.h"  
#include "ns3/simulator-module.h"  
#include "ns3/node-module.h"  
#include "ns3/helper-module.h"
```

-
- The next line in the first.cc script is a namespace declaration.

```
using namespace ns3;
```

- Logging

```
NS_LOG_COMPONENT_DEFINE ("FirstScriptExample");
```

-
- The next lines of the script you will find are,

```
int  
main (int argc, char *argv[])  
{
```

```
LogComponentEnable("UdpEchoClientApplication", LOG_LEVEL_INFO);  
LogComponentEnable("UdpEchoServerApplication", LOG_LEVEL_INFO);
```

- 1.NS_LOG_ERROR — Log error messages;
- 2.NS_LOG_WARN — Log warning messages;
- 3.NS_LOG_DEBUG — Log relatively rare, ad-hoc debugging messages;
- 4.NS_LOG_INFO — Log informational messages about program progress;
- 5.NS_LOG_FUNCTION — Log a message describing each function called;
- 6.NS_LOG_LOGIC — Log messages describing logical flow within a function;
- 7.NS_LOG_ALL — Log everything.
- 8.NS_LOG_UNCOND — Log the associated message unconditionally.

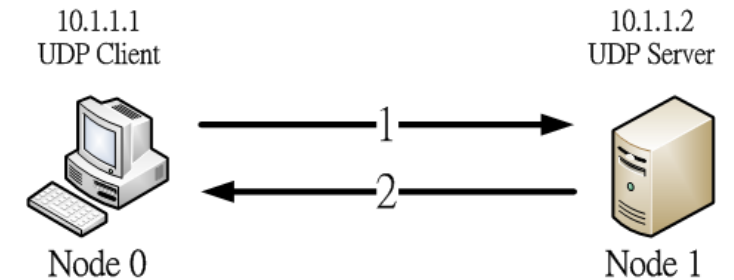
Topology Helpers

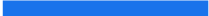
- Create the ns-3 Node objects that will represent the computers in the simulation.

- We are constructing a point-to-point link

```
NodeContainer nodes;  
nodes.Create (2);
```

```
PointToPointHelper pointToPoint;  
pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));  
pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));
```



- 
- We will need to have a list of all of the NetDevice objects that are created

```
NetDeviceContainer devices;  
devices = pointToPoint.Install (nodes);
```

Internet Stack Helper

- Protocol stacks installed on our nodes
 - TCP, UDP, IP, etc.

```
InternetStackHelper stack;  
stack.Install (nodes);
```

- Ipv4AddressHelper

- make the association between an IP address and a device

```
Ipv4AddressHelper address;  
address.SetBase ("10.1.1.0", "255.255.255.0");
```

```
Ipv4InterfaceContainer interfaces = address.Assign (devices);
```

Applications

- Two specializations of the core ns-3 class Application called
 - UdpEchoServerApplication
 - UdpEchoClientApplication
- UdpEchoServerHelper

```
UdpEchoServerHelper echoServer (9);  
  
ApplicationContainer serverApps = echoServer.Install (nodes.Get (1));  
serverApps.Start (Seconds (1.0));  
serverApps.Stop (Seconds (10.0));
```

- 
- UdpEchoClientHelper

```
UdpEchoClientHelper echoClient (interfaces.GetAddress (1), 9);  
echoClient.SetAttribute ("MaxPackets", UIntegerValue (1));  
echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.)));  
echoClient.SetAttribute ("PacketSize", UIntegerValue (1024));  
  
ApplicationContainer clientApps = echoClient.Install (nodes.Get (0));  
clientApps.Start (Seconds (2.0));  
clientApps.Stop (Seconds (10.0));
```

Simulator

- To run the simulation

scheduled events in the simulator at 1.0 seconds, 2.0 seconds and two events at 10.0 seconds

```
Simulator::Run
```

```
    Simulator::Destroy ();  
    return 0;  
}
```

PCAP file

```
////////////////////////////////////
```

```
if (tracing == true)
```

```
{
```

```
    pointToPoint.EnablePcapAll ("first");
```

```
}
```

```
////////////////////////////////////
```

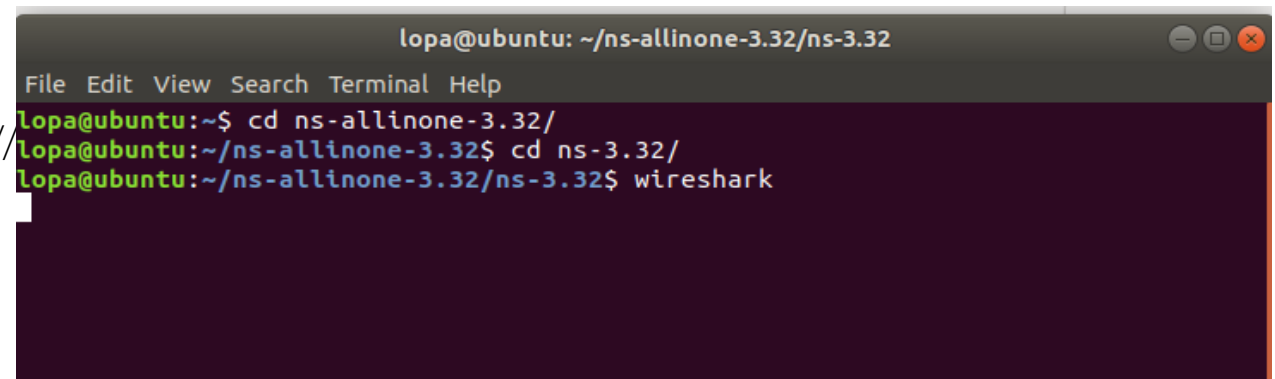
```
pointToPoint.EnablePcapAll ("p2p");// for point to point finding packet capture for all
```

```
// this pcap file will be opened using wireshark or tcpdump
```

```
csma.EnablePcap ("csma1", csmaDevices.Get (1), true);
```

```
csma.EnablePcap ("csma3", csmaDevices.Get (3), true);
```

```
////////////////////////////////////to run////////////////////////////////////
```

A terminal window titled 'lopa@ubuntu: ~/ns-allinone-3.32/ns-3.32' with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal shows three commands being executed: 'cd ns-allinone-3.32/', 'cd ns-3.32/', and 'wireshark'. The prompt changes from 'lopa@ubuntu:~\$' to 'lopa@ubuntu:~/ns-allinone-3.32\$' and finally to 'lopa@ubuntu:~/ns-allinone-3.32/ns-3.32\$' after the first two commands. The third command 'wireshark' is entered but its output is not visible.

```
lopa@ubuntu: ~/ns-allinone-3.32/ns-3.32
File Edit View Search Terminal Help
lopa@ubuntu:~$ cd ns-allinone-3.32/
lopa@ubuntu:~/ns-allinone-3.32$ cd ns-3.32/
lopa@ubuntu:~/ns-allinone-3.32/ns-3.32$ wireshark
```

TraceFile

Trace-metrics installation

sudo apt install default-jdk (for java installation)

Download tracemetrics and unzip to folder

```
////////////////////////////////////code////////////////////////////////////
```

```
AsciiTraceHelper ascii;
```

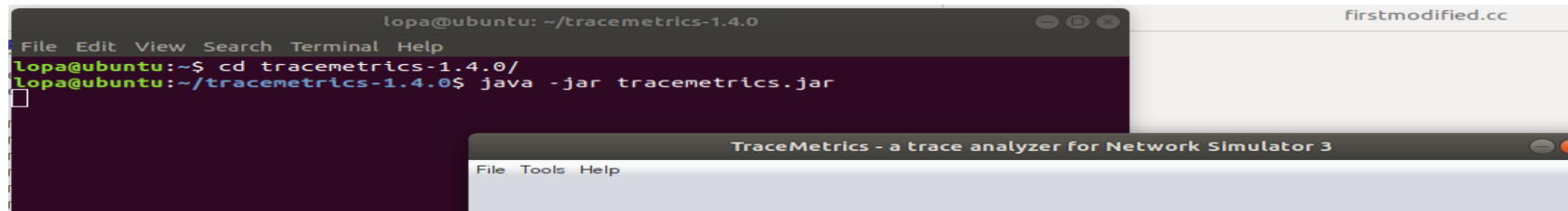
```
pointToPoint.EnableAsciiAll(ascii.CreateFileStream("first.tr"));
```

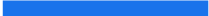
```
////////////////////////////////////
```

```
pointToPoint.EnableAsciiAll(ascii.CreateFileStream("point2point.tr"));
```

```
csma.EnableAsciiAll(ascii.CreateFileStream("csma2.tr"));
```

```
////////////////////////////////////to run////////////////////////////////////
```





NetAnim

Header file

```
#include "ns3/netanim-module.h"
```

```
////////////////////////////////////
```

```
AnimationInterface anim("first.xml"); //added
```

```
anim.SetConstantPosition(nodes.Get(0),10.0,20.0);
```

```
anim.SetConstantPosition(nodes.Get(1),20.0,30.0);
```

```
////////////////////////////////////
```

```
AnimationInterface anim("second.xml");
```

```
anim.SetConstantPosition(p2pNodes.Get(0),10.0,10.0); //either we can use p2p or csma nodes 10.0, 20.0.. are the position
```

```
anim.SetConstantPosition(p2pNodes.Get(1),20.0,20.0);
```

```
anim.SetConstantPosition(csmaNodes.Get(1),30.0,30.0);
```

```
anim.SetConstantPosition(csmaNodes.Get(2),40.0,40.0);
```

```
anim.SetConstantPosition(csmaNodes.Get(3),50.0,50.0);
```

NetAnim

